



UNINASSAU



Back-end Frameworks

Arquitetura MVC e CLIENTE-SERVIDOR

Prof^o: Joseph Donald

Contatos:

 www.josephdonald.wordpress.com

 (83) 98228-8607

 @josephdonald

 030106382@prof.uninassau.edu.br

“Se você tem uma maçã e eu tenho outra; e nós trocamos as maçãs, então cada um terá sua maçã. Mas se você tem uma ideia e eu tenho outra, e nós as trocamos; então cada um terá duas ideias.”

George Bernard Shaw



- Um padrão é uma descrição do problema e a essência da sua solução.
- Documenta boas soluções para problemas recorrentes.
- Deve ser suficientemente abstrato para ser reusado em aplicações diferentes.

- Embora cada sistema seja único, é comum que sistemas do mesmo domínio tenham arquiteturas similares.
- Um projeto pode ser baseado em um padrão arquitetural conhecido

Descrição

- Separa a apresentação e a interação dos dados do sistema, pois os três componentes tem responsabilidades distintas mas interagem entre si.
- Quando é recomendado?
 - ✓ Quando existem várias maneiras de visualizar e interagir com os dados.
 - ✓ São desconhecidos (ou são voláteis) os requisitos de interação com os dados.

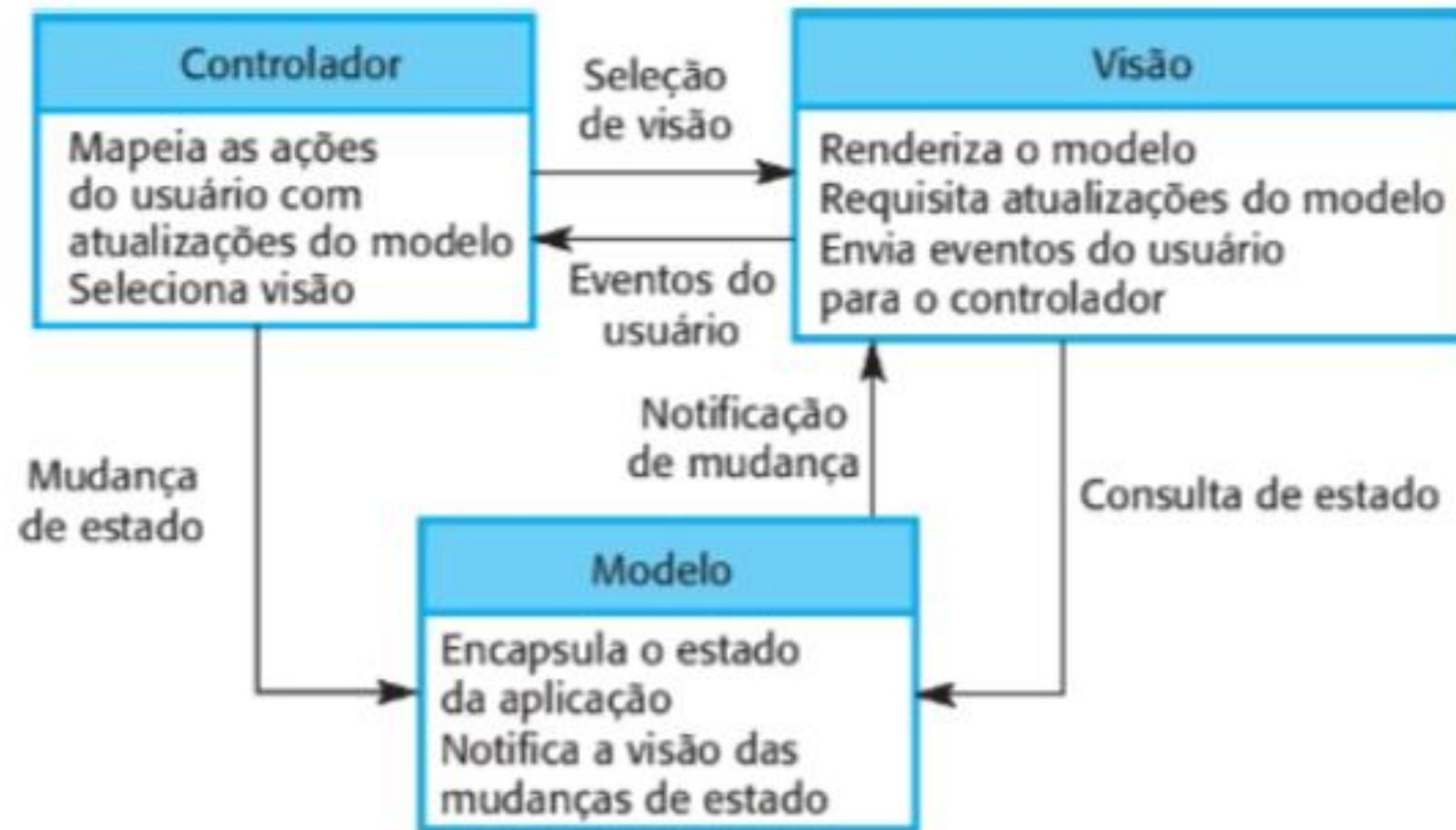
Características

- Largamente utilizado em aplicações interativas WEB
- Organiza o sistema em três componentes:
 - ✓ **Modelo (Model):** contém as funcionalidades de dados principais
 - ✓ **Visão (View):** responsável por apresentar dados ao usuário
 - ✓ **Controlador (Controller):** trata os eventos de entrada/saída

FIGURA 6.4 O padrão MVC (Modelo-Visão-Controlador).

Nome	MVC (Modelo-Visão-Controlador)
Descrição	Separa a apresentação e a interação dos dados do sistema. O sistema é estruturado em três componentes lógicos que interagem entre si. O componente Modelo gerencia os dados do sistema e as operações a eles associadas. O componente Visão define e gerencia como os dados são apresentados ao usuário. O componente Controlador gerencia a interação do usuário (por exemplo, pressionamento de teclas, cliques de mouse etc.) e passa essas interações para Visão e Modelo. Ver Figura 6.5.
Exemplo	A Figura 6.6 mostra a arquitetura de uma aplicação web, organizada com o uso do padrão MVC.
Quando é utilizado	É utilizado quando há várias maneiras de visualizar e interagir com os dados. Também é utilizado quando os requisitos futuros para interação e apresentação dos dados são desconhecidos.
Vantagens	Permite que os dados sejam alterados independentemente de sua representação e vice-versa. Apoia a apresentação dos mesmos dados de maneiras diferentes, exibindo as alterações feitas em uma representação em todas as demais.
Desvantagens	Pode envolver mais código e aumentar sua complexidade quando o modelo de dados e as interações forem simples.

FIGURA 6.5 A organização do MVC (Modelo-Visão-Controlador).





Vantagens

- Permite que os dados sejam alterados de forma independente de sua representação.
- Apoia a apresentação dos mesmos dados de maneiras diferentes
- Facilita a distribuição do componente de visão, pois os dados são mantidos centralizados e protegidos

Desvantagens

- Complexidade excessiva quando o modelo de dados e de interações é muito simples.
- A estrutura do padrão pode impor código adicional desnecessário.

Exemplos de frameworks que utilizam esse padrão:

- Django
- Ruby on Rails
- Spring Framework



Principais componentes

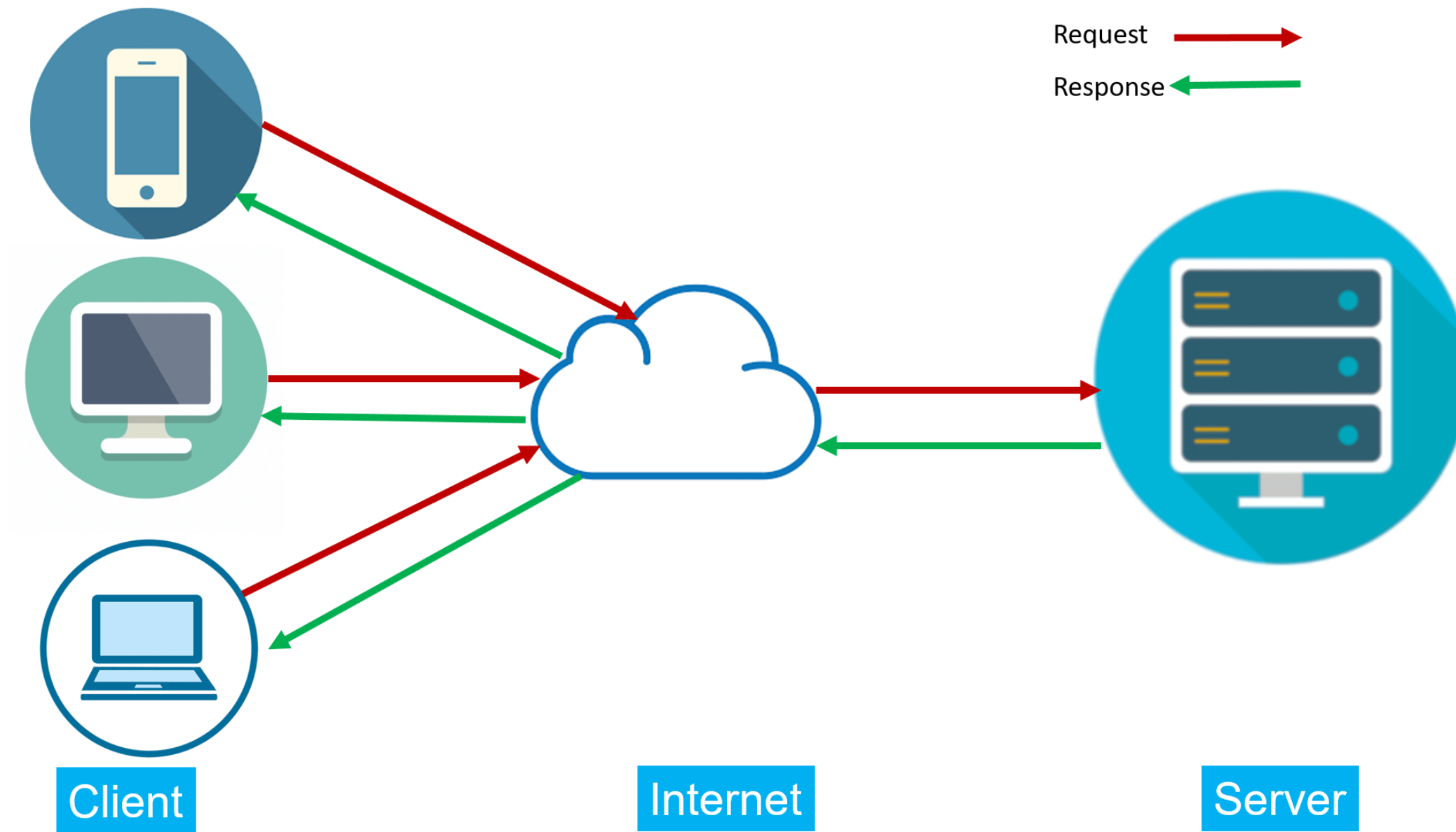
- Um conjunto de servidores que oferecem um serviço.
- Um conjunto de clientes que demandam os serviços oferecidos.
- Uma rede que permite que os clientes acessem os serviços
- Exemplos de servidores
 - ✓ Servidor WEB
 - ✓ Servidor de impressão
 - ✓ Servidor de arquivos, etc.

Características

- Requer uma estrutura de rede para clientes acessarem os serviços.
- Clientes sabem quais os serviços e servidores estão disponíveis.
- Servidores não sabem quem são os clientes.
- Essencialmente, é usado o protocolo request-reply
 1. Um cliente faz um pedido ao servidor e espera pela resposta.
 2. O servidor executa o serviço e responde ao cliente.

FIGURA 6.12 O padrão cliente-servidor.

Nome	Cliente-servidor
Descrição	Em uma arquitetura cliente-servidor, o sistema é apresentado como um conjunto de serviços, e cada serviço é fornecido por um servidor separado. Os clientes são usuários desses serviços e acessam os servidores para usá-los.
Exemplo	A Figura 6.13 é um exemplo de biblioteca de filmes e vídeos/DVDs organizada como um sistema cliente-servidor.
Quando é utilizado	Utilizado quando os dados em um banco de dados compartilhado têm de ser acessados a partir de diversos locais. Como os servidores podem ser replicados, eles também podem ser utilizados quando a carga em um sistema for variável.
Vantagens	A principal vantagem desse modelo é que os servidores podem ser distribuídos em rede. A funcionalidade geral (por exemplo, um serviço de impressão) pode estar disponível para todos os clientes e não precisa ser implementada por todos os serviços.
Desvantagens	Cada serviço é um único ponto de falha e, portanto, é suscetível a ataques de negação de serviço ou a falhas no servidor. O desempenho pode ser imprevisível porque depende da rede e também do sistema. Podem surgir problemas de gerenciamento se os servidores forem de propriedade de organizações diferentes.



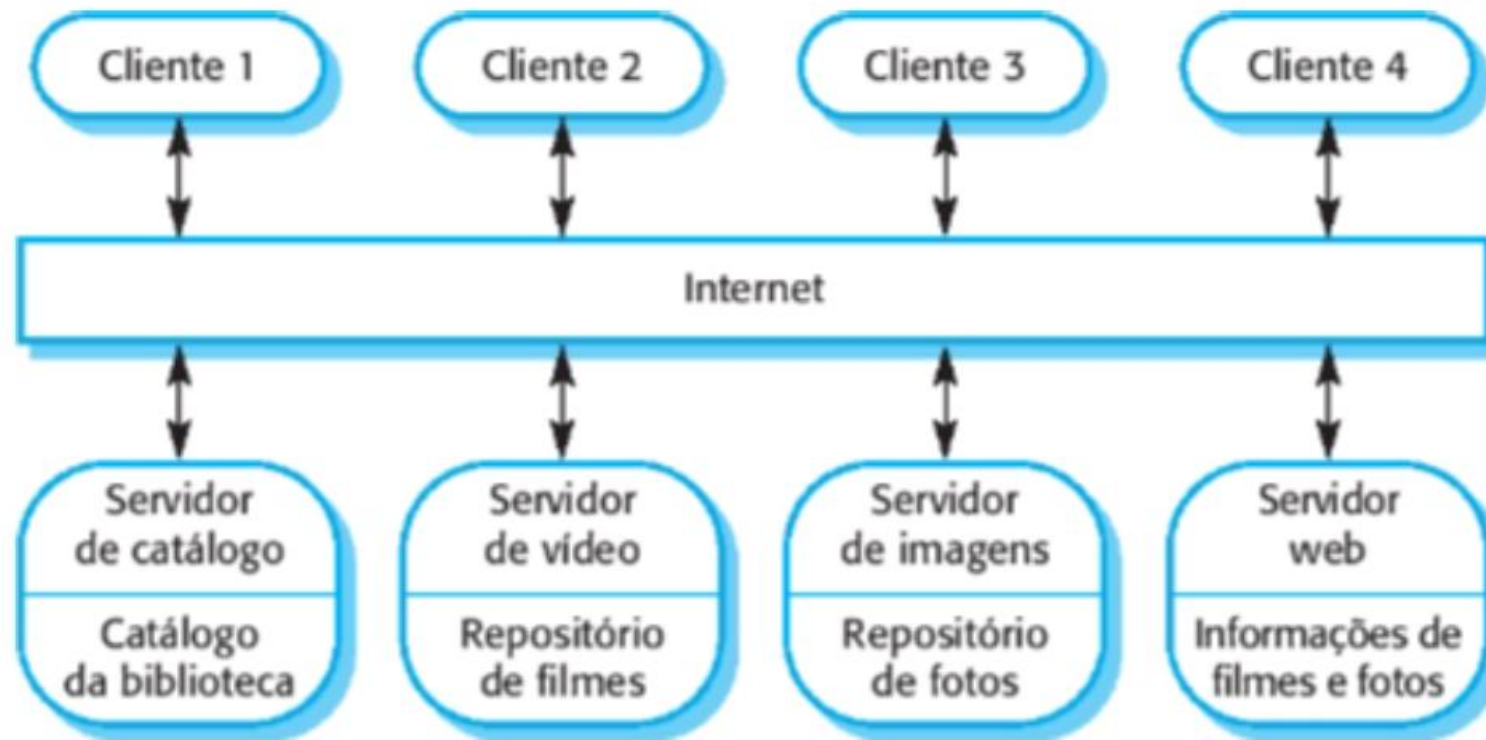
Principais componentes

- Um conjunto de servidores que oferecem um serviço.
- Um conjunto de clientes que demandam os serviços oferecidos.
- Uma rede que permite que os clientes acessem os serviços.

Arquitetura Cliente-Servidor

Exemplo

FIGURA 6.13 Arquitetura cliente-servidor de uma biblioteca de filmes.



Vantagens

- A distribuição de dados é fácil e direta
- Faz uso efetivo dos recursos em rede e possibilita hardware mais barato
- É fácil adicionar novos servidores ou atualizar servidores existentes

Desvantagens

- Não prevê modelo de dados compartilhado, pois outros subsistemas podem utilizar diferentes organizações de dados.
- Não prevê registro central de serviços, sendo difícil descobrir quais serviços estão disponíveis.

Exemplos de frameworks que utilizam esse padrão:

- Flask
- Express.js
- ASP .NET

